Human-in-the-Loop DevOps Report

Project

yatrikui

Review

Risk

Dockerfile

Okay, for a project named `yatrikui`, which sounds like a User Interface (UI) project (likely a web frontend like React, Angular, Vue, or a static site), the best practice is to use a multi-stage Dockerfile. This keeps the final image small by separating the build environment from the serving environment. Here's a general Dockerfile, assuming it's a Node.js-based frontend that builds into static files, which are then served by Nginx. --- **1. Create a `.dockerignore` file (crucial for performance and clean builds):** In the root of your `yatrikui` project, create a file named `.dockerignore` with the following content: ```dockerignore node_modules build dist .git .gitignore .env *.log Dockerfile docker-compose.yml README.md LICENSE ``` --- **2. Create a `nginx.conf` file (for Single Page Application routing):** In the root of your `yatrikui` project, create a file named `nginx.conf` with the following content. This configuration is essential for Single Page Applications (SPAs) to handle client-side routing correctly (e.g., refreshing a `/about` page should still load `index.html`). ```nginx server { listen 80; server_name localhost; root /usr/share/nginx/html; index index.html index.htm; # This is crucial for SPAs: try to serve the file directly, # if not found, fallback to index.html for client-side routing. location / { try_files $uri $uri/ /index.html; } # Optional: Cache static assets for better performance location ~* \.(js|css|png|jpg|jpeg|gif|ico|woff|woff2|ttf|svg|eot)$ { expires 1y; add_header Cache-Control "public, immutable"; } error_page 500 502 503 504 /50x.html; location = /50x.html { root /usr/share/nginx/html; } } ``` --- **3. Create the `Dockerfile`:** In the root of your `yatrikui` project, create a file named `Dockerfile` with the following content: ```dockerfile # Stage 1: Build the frontend application # Use a Node.js base image for building. Using alpine for smaller image size. FROM node:20-alpine AS builder # Set the working directory inside the container WORKDIR /app # Copy package.json and package-lock.json (or yarn.lock) first. # This allows Docker to cache the npm install step if only source files change. COPY package*.json ./ # Install project dependencies. # 'npm ci' is preferred over 'npm install' in CI/CD environments for reproducible builds. RUN npm ci --omit=dev # Copy the rest of the application source code COPY . . # Build the frontend application. # Replace 'npm run build' with your actual build command if it's different. # The output (static files) is typically placed in a 'build' or 'dist' directory. # We'll assume 'build' here, which is common for Create React App, etc. RUN npm run build # Stage 2: Serve the built application with Nginx # Use a lightweight Nginx base image for serving static files FROM nginx:alpine # Copy the custom Nginx configuration file COPY nginx.conf /etc/nginx/conf.d/default.conf # Copy the built static files from the builder stage into Nginx's web root # Ensure '/app/build' matches the output directory of your build command in Stage 1. COPY --from=builder /app/build /usr/share/nginx/html # Expose port 80, which Nginx listens on EXPOSE 80 # The default command for nginx:alpine starts Nginx, so we don't need a CMD instruction. # CMD ["nginx", "-g", "daemon off;"] # This is the default, no need to specify usually. ``` --- **How to Build and Run:** 1. **Build the Docker image:** ```bash docker build -t yatrikui:latest . ``` 2. **Run the Docker container:** ```bash docker run -p 80:80 yatrikui:latest ``` This will map port 80 on your host to port 80 inside the container. You can then access your application in your browser at

`http://localhost`. --- **Explanation and Customization:** * **`node:20-alpine`**: You can change the Node.js version (`18`, `16`, etc.) or even use a full Node.js image if `alpine` causes issues with some native dependencies (though rare for frontends). * **`npm ci --omit=dev`**: This command is great for CI/CD as it uses `package-lock.json` to install exact dependency versions. `--omit=dev` prevents installing `devDependencies` which are not needed for the final production build. If you use `yarn`, replace `npm ci` with `yarn install --frozen-lockfile --production`. * **`npm run build`**: This is the standard command for most modern frontend frameworks. If your project uses a different command (e.g., `ng build --configuration production` for Angular, `vite build` for Vite), adjust this line. * **`/app/build`**: This path assumes your build output directory is named `build`. Common alternatives are `dist`, `out`, `public`. Make sure this matches where your `npm run build` command places the static files. * **`nginx.conf`**: This is crucial for SPAs. If your project is not an SPA and serves multiple HTML files directly, you might simplify or remove the `try_files` line in `nginx.conf`. * **`EXPOSE 80`**: Nginx usually listens on port 80. If you configure Nginx to listen on a different port, update this. This multi-stage Dockerfile provides a robust, efficient, and small image for deploying your `yatrikui` application.

Kubernetes

Okay, here's a basic Kubernetes Deployment YAML for your `yatrikui` project. **Important Notes before you use it:** 1. **Image Name:** You **MUST** replace `yatrikui:latest` with the actual name and tag of your Docker image. For example, `yourdockerhubusername/yatrikui:v1.0.0` or `your-private-registry.com/yatrikui:production`. 2. **Container Port:** The `containerPort: 80` assumes your `yatrikui` application listens on port 80 inside the container. If it listens on a different port (e.g., 3000, 8080), update this value. 3. **Resources:** The `resources` section provides default CPU and memory requests and limits. Adjust these based on the actual requirements of your `yatrikui` application. 4. **Health Checks:** I've commented out `livenessProbe` and `readinessProbe`. It's highly recommended to uncomment and configure these with actual health check endpoints from your `yatrikui` application to ensure Kubernetes can properly manage your application's health. 5. **Service & Ingress:** This is just a Deployment. To make your `yatrikui` application accessible from outside the cluster, you'll typically need a Kubernetes `Service` (e.g., of type `LoadBalancer` or `NodePort`) and possibly an `Ingress`. I'll provide a basic `Service` example below the Deployment. --- ### 1. `yatrikui-deployment.yaml` ```yaml # Kubernetes Deployment for the yatrikui application apiVersion: apps/v1 kind: Deployment metadata: name: yatrikui-deployment # Name of the Deployment labels: app: yatrikui # Labels to identify this Deployment spec: replicas: 1 # Number of desired pod replicas. Increase for high availability. selector: matchLabels: app: yatrikui # Selector to find pods managed by this Deployment template: # Pod template definition metadata: labels: app: yatrikui # Labels for the pods created by this Deployment spec: containers: - name: yatrikui-container # Name of the container # IMPORTANT: Replace 'yatrikui:latest' with your actual Docker image name and tag. # Example: yourdockerhubusername/yatrikui:v1.0.0 # Example for a UI: nginx:latest or a custom image that serves your UI files image: yatrikui:latest ports: - containerPort: 80 # The port your application listens on inside the container name: http # Optional: give a name to the port # Optional: Define resource requests and limits. # This helps Kubernetes schedule your pods efficiently and prevents resource exhaustion. resources: requests: cpu: "250m" # 0.25 CPU core memory: "256Mi" # 256 MiB of memory limits: cpu: "500m" # 0.5 CPU core memory: "512Mi" # 512 MiB of memory # Optional: Environment variables # env: # - name: API_BASE_URL # value: "http://backend-service.default.svc.cluster.local" # Example for a UI talking to a backend # Optional: Liveness and Readiness Probes for health checks. Highly recommended! # livenessProbe: # httpGet: # path: /healthz # Replace with your actual health check endpoint (e.g., /ping, /status) # port: 80 # initialDelaySeconds: 15 # Wait 15 seconds before first probe # periodSeconds: 20 # Probe every 20 seconds # timeoutSeconds: 5 # Timeout if no response in 5 seconds # failureThreshold: 3 # Consider failed after 3 consecutive failures # readinessProbe: # httpGet: # path: /ready # Replace with your actual readiness check endpoint # port: 80 # initialDelaySeconds: 5 # Wait 5 seconds before first probe # periodSeconds: 10 # Probe every 10 seconds # timeoutSeconds: 3 # Timeout if no response in 3 seconds # failureThreshold: 3 # Consider not ready after 3 consecutive failures ``` --- ### 2. Basic `yatrikui-service.yaml` (To expose your application) After deploying your `yatrikui` application, you'll need a Kubernetes

Service to expose it. Here are two common types: #### Option A: `ClusterIP` (Internal access only, useful if you have an Ingress) ```yaml # Kubernetes Service for yatrikui (ClusterIP - internal access) apiVersion: v1 kind: Service metadata: name: yatrikui-service # Name of the Service labels: app: yatrikui spec: selector: app: yatrikui # Selects pods with the label 'app: yatrikui' ports: - protocol: TCP port: 80 # The port the Service itself will listen on targetPort: 80 # The port on the container that the Service will forward traffic to (must match containerPort in Deployment) type: ClusterIP # Exposes the Service on an internal IP in the cluster ``` #### Option B: `LoadBalancer` (External access via cloud provider's load balancer) ```yaml # Kubernetes Service for yatrikui (LoadBalancer - external access) apiVersion: v1 kind: Service metadata: name: yatrikui-service # Name of the Service labels: app: yatrikui spec: selector: app: yatrikui # Selects pods with the label 'app: yatrikui' ports: - protocol: TCP port: 80 # The port the Service itself will listen on targetPort: 80 # The port on the container that the Service will forward traffic to type: LoadBalancer # Creates a cloud provider specific load balancer to expose the Service externally ``` --- ### How to use these files: 1. **Save:** * Save the Deployment YAML as `yatrikui-deployment.yaml`. * Save your chosen Service YAML (e.g., `LoadBalancer` type) as `yatrikui-service.yaml`. 2. **Customize:** **Crucially, edit `yatrikui-deployment.yaml`** to replace `image: yatrikui:latest` with your actual image. Adjust ports and resources if needed. 3. **Apply:** ```bash kubectl apply -f yatrikui-deployment.yaml kubectl apply -f yatrikui-service.yaml ``` 4. **Verify:** ```bash kubectl get deployments kubectl get pods -l app=yatrikui kubectl get services ``` If you used `LoadBalancer` type, `kubectl get services` will eventually show an external IP or hostname that you can use to access your `yatrikui` application. If you used `ClusterIP`, you'd typically need an Ingress resource or port-forwarding to access it externally.

CI/CD

Okay, for a project named `yatrikui`, which sounds like a User Interface (UI) project, likely using JavaScript/TypeScript with a framework like React, Vue, Angular, or a build tool like Vite/Webpack, here's a comprehensive GitHub Actions workflow. This workflow will: 1. **Trigger** on pushes to `main` and `develop` branches, and on all Pull Requests. 2. **Install Node.js** and project dependencies. 3. **Run linting** checks (if configured). 4. **Build** the project. 5. **Run tests** (unit/integration). 6. **Upload build artifacts** for later inspection or deployment. --- ### Workflow File: `.github/workflows/ci.yml` ```yaml name: CI/CD Pipeline for yatrikui on: push: branches: - main - develop pull_request: branches: - main - develop workflow_dispatch: # Allows manual triggering of the workflow jobs: build-and-test: name: Build and Test runs-on: ubuntu-latest # Use the latest Ubuntu runner steps: - name: Checkout Repository uses: actions/checkout@v4 # Action to checkout your code - name: Setup Node.js uses: actions/setup-node@v4 # Action to set up Node.js environment with: node-version: '20' # Specify the Node.js version (LTS recommended) cache: 'npm' # Cache npm dependencies for faster installs. Can be 'yarn' or 'pnpm' - name: Install Dependencies run: npm ci # 'npm ci' is preferred in CI environments for clean, reproducible installs - name: Run Lint Checks # This step assumes you have a 'lint' script defined in your package.json # e.g., "lint": "eslint . --ext .ts,.tsx,.js,.jsx" run: npm run lint env: CI: true # Some linters might behave differently in CI (e.g., exit on warnings) - name: Build Project # This step assumes you have a 'build' script defined in your package.json # e.g., "build": "vite build" or "build": "react-scripts build" run: npm run build env: CI: true # Often used by build tools - name: Run Tests # This step assumes you have a 'test' script defined in your package.json # e.g., "test": "jest" or "test": "react-scripts test" run: npm run test env: CI: true # Essential for many test runners (e.g., Jest, Vitest) to run tests in non-interactive mode - name: Upload Build Artifacts uses: actions/upload-artifact@v4 # Action to upload built files as an artifact with: name: yatrikui-build-artifacts path: dist # Adjust this path to your actual build output directory (e.g., build/, out/, public/) retention-days: 5 # How long to keep the artifact ``` --- ### How to Use This Workflow: 1. **Create the Directory:** In your `yatrikui` project repository, create a directory structure: `.github/workflows/`. 2. **Save the File:** Save the YAML content above as `ci.yml` (or any other `.yml` name) inside the `.github/workflows/` directory. * Example path: `your-repo/.github/workflows/ci.yml` 3. **Ensure `package.json` Scripts:** This workflow heavily relies on standard npm scripts. Make sure your `package.json` includes: * `"scripts": { ... }` * `"lint": "..."` (e.g., using ESLint) * `"build": "..."` (e.g., using Vite, Webpack, create-react-app, Vue CLI) * `"test": "..."` (e.g., using Jest, Vitest, React Testing Library) * If you

don't have one of these, you can either omit the corresponding step in the workflow or add the script to your `package.json`. 4. **Adjust `node-version`:** Change `'20'` to your preferred Node.js LTS version if different. 5. **Adjust `cache`:** If you use Yarn or pnpm, change `cache: 'npm'` to `cache: 'yarn'` or `cache: 'pnpm'` respectively. 6. **Adjust `path` for Artifacts:** Make sure `path: dist` matches the actual output directory of your build process (e.g., `build`, `out`, `public`). 7. **Commit and Push:** Push these changes to your GitHub repository. The workflow will automatically trigger on the specified branches and pull requests. This setup provides a robust base for continuous integration, ensuring that your `yatrikui` project is always buildable and testable.